Language Agent Basics

Jeremy, Ashley, Christopher

Gap between LLMS and Intelligent Agents

What LLMs are good at:

- Language understanding, generation, summarization
- Few-shot and zero-shot reasoning across diverse tasks
- Code generation, translation, question answering
- Emergent chain-of-thought reasoning at scale

What LLMs fundamentally cannot do:

- Access real-time or post-training information
- Perform reliable arithmetic or precise calculations
- Take actions in the world (search, click, execute code)
- Remember past failures and improve from them
- Verify their own outputs against ground truth

Real World Implications

Why this matters now:

- LLMs are increasingly being deployed as **agents**, not just text generators
- Real-world tasks (booking, coding, research, navigation) require planning, tool use, and recovery from failure
- The gap between "impressive demo" and "reliable autonomous system" is precisely these limitations

How do we turn a language model into an agent that can reason about what it doesn't know, act in the world, use external tools, and get better over time?

Overview: Each paper addresses a gap

- **ReAct: Reasoning-action gap** — reasoning without grounding hallucinates; action without reasoning gets stuck
- **Toolformer: Knowledge gap** — model's training data is static, wrong on facts/math
- **Reflexion: Learning gap** — agents fail and have no way to recover or improve

ReAct: Synergizing Reasoning and Acting in Language Models

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, Yuan Cao

Reasoning and Acting in Tasks

Imagine cooking a dish. We use reasoning and acting to guide us through the process.

Reasoning:

- Track progress - “Now that I have peeled the vegetables, I should dice them.”
- Adjust plans - “I have no garlic. I should substitute it with garlic powder.”
- Acknowledge when external information is needed - “How long should I boil potatoes to make them tender? I should look this up.”

Acting:

- Open the cookbook and read the recipes, open the fridge and look at the ingredients, …
- Can support reasoning - “With the ingredients I have, what dish can I make?”

Reasoning and acting allow humans to learn and adapt to new tasks quickly.

Prior Work

- “Chain-of-thought” reasoning with LLMs have shown to be effective
  - However, these models use their own internal representations that are not grounded in the real world

- Other work has shown that pre-trained language models can be used for planning and acting in interactive environments
  - However, these studies do not use the LMs for abstract high-level reasoning or to maintain a working memory to support acting

- No prior work had studied how reasoning and acting can be combined to improve performance in general tasks

Failure of older models

- This model fails to finish the task since it requires complex reasoning over the trajectory of its context
- This model fails due to hallucinating actions

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

Answer: “keyboard function keys”

(1c) Act-Only

Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control …

Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: …

Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software …

Act 4: Finish[yes]

(2) AlfWorld

You are in the middle of a room. Looking quickly around you, you see a cabinet 6, a cabinet 1, a coffee machine 1, a countertop 3, a stove burner 1, and a toaster 1.
Your task is to: Put some pepper shaker on a drawer.

(2a) Act-Only

Act 1: Go to drawer 1
Obs 1: The drawer 1 is closed.

Act 2: Open drawer 1
Obs 2: You open the drawer 1. The drawer 1 is open. In it, you see a dishsponge 2, and a spoon 1.

Act 3: Go to sinkbasin 1
Obs 3: On the sinkbasin 1, you see a dishsponge 3, a spatula 1, and a spoon 2.

Act 4: Take peppershaker 1 from sinkbasin 1
Obs 4: Nothing happens.

Act 5: Take peppershaker 1 from sinkbasin 1
Obs 5: Nothing happens.

ReAct: Reasoning + Acting

Main Idea: Agent's action space is now possible actions + language $\hat{\mathcal{A}} = \mathcal{A} \cup \mathcal{L}$

Thought/Reasoning Trace: $\hat{a}_t \in \mathcal{L}$ - an element of the language space that uses its current context to find useful information then update the context with this new information

(2) AlfWorld

You are in the middle of a room. Looking quickly around you, you see a cabinet 6, a cabinet 1, a coffee machine 1, a countertop 3, a stove burner 1, and a toaster 1.
Your task is to: Put some pepper shaker on a drawer.

(2b) ReAct (Reason + Act)

Act 1: Think[First I need to find a pepper shaker...more likely to appear in cabinets (1-6), countertops (1-3), ...]
Act 2: Go to cabinet 1
Obs 2: On the cabinet 1, you see a vase 2.

(Here the agent go to cabinet 1, then cabinet 2, 3, then countertop 1 and 2)

Act 6: Go to countertop 3
Obs 6: On the countertop 3, you see a apple 1, a bread 1, ... a pepper shaker 1, and a vase 3.

Act 7: Take pepper shaker 1 from countertop 3
Obs 7: You pick up the pepper shaker 1 from the countertop 3.

Act 8: Think[Now I find a pepper shaker 1. Next, I need to put it in/on drawer 1.]
Act 9: Go to drawer 1
Obs 9: Drawer 1 is closed.

Act 10: Open drawer 1
Obs 10: You open Drawer 1 …

Act 11: Put pepper shaker 1 in/on drawer 1
Obs 11: You put pepper shaker 1 in/on the drawer 1. ✓

Model Setup

- The authors use a frozen LLM, PaLM-540B, since learning in the augmented action space is difficult due to the unlimited size of the language space

- They prompt the LLM with few-shot in-context examples to produce the possible actions and free-form language thoughts for the task
  - These examples demonstrate a human's thought process, actions, and observations to successfully complete the given task

- Frequency of thought vs. action generation depends on the type of task
  - Reasoning-intensive task vs. decision making

Knowledge-Intensive Reasoning Tasks

Goal: Answer a question/validate a fact using multi-hop reasoning

- **Reasoning:** Use a Wikipedia API to find relevant information towards the answer
- **Acting:** Search for a page on Wikipedia, lookup keywords on a page, finalize answer

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

Q1: What program was the Apple remote designed to interact with? → [ANSWER1]

Q2: What other device can control [ANSWER1] → [ANSWER2] (final answer)

KIRT: Datasets for Testing

1. **HotpotQA:** Multi-hop question answering dataset that requires reasoning from two or more Wikipedia passages to answer

Question	Musician and satirist Allie Goertz wrote a song about the "The Simpsons" character Milhouse, who Matt Groening named after who?

2. **FEVER:** Dataset that contains claims that are labeled “SUPPORTS”, “REFUTES”, or “NOT ENOUGH INFO” based on if there exists a Wikipedia passage to support the claim

Original	Determine if there is Observation that SUPPORTS or REFUTES a Claim, or if there is NOT ENOUGH INFORMATION.
Claim	Nikolaj Coster-Waldau worked with the Fox Broadcasting Company.
Answer	SUPPORTS

KIRT: ReAct Action Space

1. **search**[*entity*]: retrieves the first 5 sentences from the Wikipedia page for *entity* if it exists. Otherwise, return the five most similar entities according to the Wikipedia search engine

2. **lookup**[*string*]: Find next sentence on the page that contains *string* (similar to Ctrl+F)

3. **finish**[*answer*]: submit *answer* as final answer

KIRT: ReAct Prompting

For HotpotQA, choose 6 random cases from training set. For FEVER, choose 3 random cases from training set. Then, manually construct example human trajectories.

Example prompt for HotPotQA

Question	Musician and satirist Allie Goertz wrote a song about the "The Simpsons" character Milhouse, who Matt Groening named after who?
Thought 1	The question simplifies to "The Simpsons" character Milhouse is named after who. I only need to search Milhouse and find who it is named after.
Action 1	Search[Milhouse]
Observation 1	Milhouse Mussolini Van Houten is a recurring character in the Fox animated television series The Simpsons voiced by Pamela Hayden and created by Matt Groening.
Thought 2	The paragraph does not tell who Milhouse is named after, maybe I can look up "named after".
Action 2	Lookup[named after]
Observation 2	(Result 1 / 1) Milhouse was named after U.S. president Richard Nixon, whose middle name was Milhous.
Thought 3	Milhouse was named after U.S. president Richard Nixon, so the answer is Richard Nixon.
Action 3	Finish[Richard Nixon]

KIRT: Other Baselines

Standard: No thoughts, actions, or observations

Question	Musician and satirist Allie Goertz wrote a song about the "The Simpsons" character Milhouse, who Matt Groening named after who?
Answer	Richard Nixon

Chain-of-Thought (CoT): No actions or observations

Question	Musician and satirist Allie Goertz wrote a song about the "The Simpsons" character Milhouse, who Matt Groening named after who?
Thought	Let's think step by step. Milhouse was named after U.S. president Richard Nixon, so the answer is Richard Nixon.
Answer	Richard Nixon

CoT-Self Consistency (CoT-SC): Run multiple CoT models; output majority answer

KIRT: Other Baselines

Acting-Only: No thoughts

Question	Musician and satirist Allie Goertz wrote a song about the "The Simpsons" character Milhouse, who Matt Groening named after who?
Action 1	Search[Milhouse]
Observation 1	Milhouse Mussolini Van Houten is a recurring character in the Fox animated television series The Simpsons voiced by Pamela Hayden and created by Matt Groening.
Action 2	Lookup[named after]
Observation 2	(Result 1 / 1) Milhouse was named after U.S. president Richard Nixon, whose middle name was Milhous.
Action 3	Finish[Richard Nixon]

KIRT: Finetuning

Challenge: manually creating each prompt becomes unreasonable with scale

Solution (Bootstrapping Approach): Generate 3000 correct answers and trajectories with ReAct (from original manual prompts). These are used to train and finetune smaller LMs (e.g. PaLM-8/62B) so that they learn the ReAct trajectories. We can then use the smaller LMs to answer the questions.

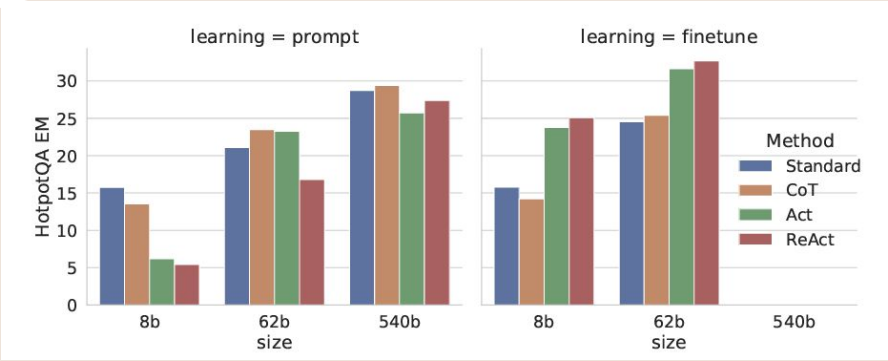
KIRT: Initial Results

Main Takeaways:

- ReAct consistently outperforms Act
- CoT-SC outperforms ReAct in HotpotQA and performs similarly to ReAct at FEVER
- ReAct is the best baseline for finetuning

Prompt Method[a]	HotpotQA (EM)	Fever (Acc)
Standard	28.7	57.1
CoT (Wei et al., 2022)	29.4	56.3
CoT-SC (Wang et al., 2022a)	33.4	60.4
Act	25.7	58.9
ReAct	27.4	60.9

Results with PaLM-540B prompting



Scaling results for HotpotQA

KIRT: ReAct vs. CoT

- CoT suffers from hallucinations but is better at reasoning structure
- The reason/action/observation cycle grounds ReAct's outputs, but this cycle impedes reasoning flexibility
  - ReAct also sometimes struggles with inferring what the next action should be
- Sometimes, ReAct performs an unhelpful search which can derail the rest of the reasoning
- **Solution:** Combine ReAct and CoT

	Type	Definition	ReAct	CoT
Success	True positive	Correct reasoning trace and facts	94%	86%
	False positive	Hallucinated reasoning trace or facts	6%	14%
Failure	Reasoning error	Wrong reasoning trace (including failing to recover from repetitive steps)	47%	16%
	Search result error	Search return empty or does not contain useful information	23%	-
	Hallucination	Hallucinated reasoning trace or facts	0%	56%
	Label ambiguity	Right prediction but did not match the label precisely	29%	28%

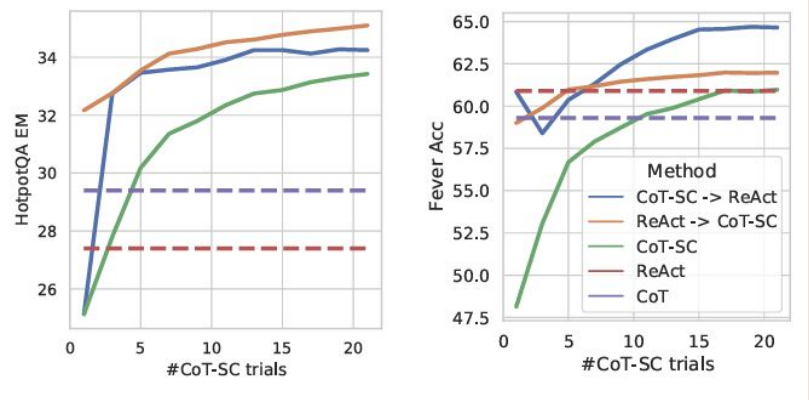
(From 50 random successes and failures from both ReAct and CoT)

KIRT: Full Results

CoT-SC → ReAct: out of the n CoT-SC samples, if the majority answer appears in less than n/2 of the samples, switch to ReAct

ReAct → CoT-SC: if ReAct fails to produce an answer within a given number of steps (7 for HotpotQA; 5 for FEVER), switch to CoT-SC

Prompt Method[a]	HotpotQA (EM)	Fever (Acc)
Standard	28.7	57.1
CoT (Wei et al., 2022)	29.4	56.3
CoT-SC (Wang et al., 2022a)	33.4	60.4
Act	25.7	58.9
ReAct	27.4	60.9
CoT-SC → ReAct	34.2	**64.6**
ReAct → CoT-SC	**35.1**	62.0
Supervised SoTA[b]	67.5	89.5



Decision Making Tasks

Two different tasks that involve complex environments, actions over long horizons, and sparse rewards:

1. **ALFWorld:** Perform a high-level task by navigating and interacting with a household
2. **WebShop:** Navigate a website and purchase an item that meets certain specifications

Note: As opposed to alternating between thoughts and actions, thoughts appear more sparsely as the agent will attempt to complete a subtask for a few actions before generating a new thought

Decision Making Tasks: ALFWorld

- Includes 6 different high-level tasks
  - Ex: find a particular item in the house and move it to a specific location
- Can contain 50+ locations and take 50+ steps to solve
- Tests the agent's ability to create subgoals, track progress, and systematically explore
- For each task, manually generate three trajectories
- **Actions:** navigate to and interact with various items around the house

Prompting Goals:

1. Decompose goal into subgoals
2. Track subgoal completion
3. Determine next subgoal
4. Use commonsense reasoning to intuit locations of items and what to do with them

Decision Making Tasks: ALFWorld

```
You are in the middle of a room.  Looking quickly around you, you see a cabinet
13, a cabinet 12, a cabinet 11, a cabinet 10, a cabinet 9, a cabinet 8, a cabinet
7, a cabinet 6, a cabinet 5, a cabinet 4, a cabinet 3, a cabinet 2, a cabinet
1, a coffeemachine 1, a countertop 1, a diningtable 1, a drawer 1, a fridge 1, a
garbagecan 1, a microwave 1, a shelf 3, a shelf 2, a shelf 1, a sinkbasin 1, a
stoveburner 4, a stoveburner 3, a stoveburner 2, a stoveburner 1, and a toaster 1.
Your task is to:  put a clean lettuce in diningtable.
> think:  To solve the task, I need to find and take a lettuce, then clean it with
sinkbasin, then put it in diningtable.
OK.
> think:  First I need to find a lettuce.  A lettuce is more likely to appear in
fridge (1), diningtable (1), sinkbasin (1), stoveburner (1-3), cabinet (1-13).  I
can check one by one, starting with fridge 1.
OK.
> go to fridge 1
The fridge 1 is closed.
> open fridge 1
```

Decision Making Tasks: ReAct-IM

Internal Monologue (IM): thoughts are limited to observations in the current environment and subgoals to complete the task

```
You are in the middle of a room.  Looking quickly around you, you see a cabinet
13, a cabinet 12, a cabinet 11, a cabinet 10, a cabinet 9, a cabinet 8, a cabinet
7, a cabinet 6, a cabinet 5, a cabinet 4, a cabinet 3, a cabinet 2, a cabinet
1, a coffeemachine 1, a countertop 1, a diningtable 1, a drawer 1, a fridge 1, a
garbagecan 1, a microwave 1, a shelf 3, a shelf 2, a shelf 1, a sinkbasin 1, a
stoveburner 4, a stoveburner 3, a stoveburner 2, a stoveburner 1, and a toaster 1.
Your task is to:  put a clean lettuce in diningtable.
> think:  To solve the task, I need to find and take a lettuce, then clean it with
sinkbasin, then put it in diningtable.
OK.
> think:  First I need to find a lettuce.
OK.
> go to fridge 1
The fridge 1 is closed.
> think:  First I need to find a lettuce.
OK.
> open fridge 1
You open the fridge 1.  The fridge 1 is open.  In it, you see a cup 3, a egg 2, a
potato 3, and a potato 2.
> think:  First I need to find a lettuce.
```

Decision Making Tasks: WebShop

- Includes 1.18 million real-world products and 12,000 human instructions
- Contains structured and unstructured text
- Agent must buy a product based on the specifications from the instruction
  - Ex: “I want a wooden chair with a white cushion costing less than $100”
- Performance is based on average score and success rate over 500 instructions
  - Average score = average % of specifications met over all episodes
  - Success = all specifications met
- **Actions:** search, choose product, choose options, buy

Prompting Goals:

1. Reason what items to explore
2. Reason when to buy an item
3. Reason what product options best meet the specifications

Decision Making Tasks: WebShop

Instruction: i would like a 3 ounce bottle of bright citrus deodorant for sensitive skin, and price lower than 50.00 dollars

Action: **search**[3 ounce bright citrus deodorant sensitive skin
Observation:
[Back to Search]
Page 1 (Total results: 50)
[Next]

[B078GWRC1J]
Bright Citrus Deodorant by Earth Mama | Natural and Safe for Sensitive Skin, Pregnancy and Breastfeeding, Contains Organic Calendula 3-Ounce
$10.99
[B078GTKVXY]
Ginger Fresh Deodorant by Earth Mama | Natural and Safe for Sensitive Skin, Pregnancy and Breastfeeding, Contains Organic Calendula 3-Ounce
$10.99
[B08KBVJ4XN]
Barrel and Oak - Aluminum-Free Deodorant, Deodorant for Men , Essential Oil-Based Scent, 24-Hour Odor Protection, Cedar & Patchouli Blend, Gentle on Sensitive Skin (Mountain Sage, 2.7 oz, 2-Pack)
$15.95

Action: **think**[B078GWRC1J and B078GTKVXY are bright citrus deodorant less then 50 dollars. I can check B078GWRC1J first.
Observation: OK.

Decision Making Tasks: Results

- BUTLER is a imitation learning agent trained on 10^5 expert trajectories
- For ALFWorld, ReAct consistently outperforms Act
  - Act struggled with decomposing into subgoals and tracking current state of the environment
- ReAct outperforms ReAct-IM on 5 out of 6 tasks
  - IM limits observations to the environment, so ReAct has more diverse reasoning

Method	Pick	Clean	Heat	Cool	Look	Pick 2	All
Act (best of 6)	88	42	74	67	72	**41**	45
ReAct (avg)	65	39	83	76	55	24	57
ReAct (best of 6)	**92**	58	**96**	86	**78**	**41**	**71**
ReAct-IM (avg)	55	59	60	55	23	24	48
ReAct-IM (best of 6)	62	**68**	87	57	39	33	53
BUTLER$_g$ (best of 8)	33	26	70	76	17	12	22
BUTLER (best of 8)	46	39	74	**100**	22	24	37

Results of ALFWorld Task

Decision Making Tasks: Results

- IL is an Imitation Learning method
- IL + RL = imitation learning + reinforcement learning
- ReAct consistently outperforms the other agents
- ReAct performed well at identifying products that met the specifications despite noisy information
  - Ex: “I want a space-saving ottoman for living room”
  - Thought: “the item that has options ‘39x18x18inch’ and ‘blue’ seems good to buy”
- Human experts tend to explore more products

Method	Score	SR
Act	62.3	30.1
ReAct	**66.6**	**40.0**
IL	59.9	29.1
IL+RL	62.4	28.7
Human Expert	82.1	59.6

Results of WebShop Task

Advantages of ReAct

1. **Intuitive and Easy to Design:** Inputted prompts can just be humans writing down their thought process with their actions

2. **General and Flexible:** Can easily customize thought space and frequency of thoughts and actions depending on the task

3. **Performant and Robust:** Boasts high performance even with few in-context prompts

4. **Human Aligned and Controllable:** Can see the model's thought process and reasoning to find errors and adjust behavior

Conclusion

Limitations:

- Heavily relies on quality of the prompt engineering
- Some tasks may require large amounts of queries to the LLM, which could be inefficient
- For the finetuning, errors in the large model could propagate to the smaller models

Future Work:

- How to optimize the quality of the human-made prompts
- Apply ReAct to multitasking
- Combining ReAct with reinforcement learning could produce stronger agents

Toolformer: Language Models Can Teach Themselves to Use Tools

Timo Schick et al. (Meta AI Research)

Strength & Weaknesses of LLMs Prior

Strengths of LLMs

- Zero shot learning
- Few shot learning
- Emergent Abilities
- Strong language understanding

Weaknesses of LLMs

- No access to up to date knowledge
- Hallucination of incorrect facts
- Poor performance on low resource languages
- Weak mathematical reasoning
- No awareness of current time

Research Gap (Limitations of Existing Tool Approaches)

- Human Annotation
- Task Specific Tool Use

Key Idea: Teaching LLMs to Use Tools

Goal: Enable language models to autonomously use external tools

Model should learn

- When to call a tool
- Which tool to use
- Which arguments to pass
- How to incorporate the result

Tool use is learned automatically using self-supervised training. (no human required)

The New England Journal of Medicine is a registered trademark of [QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society] the MMS.

Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

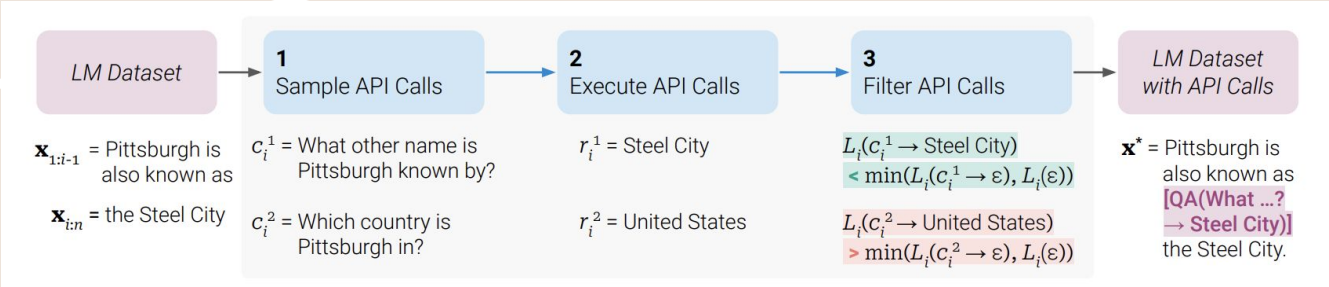
The name derives from "la tortuga", the Spanish word for [MT("tortuga") → turtle] turtle.

The Brown Act is California's law [WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.] that requires legislative bodies, like city councils, to hold their meetings open to the public.

Toolformer Training Pipeline (Overview)

How Toolformer Learns to Use Tools

1. **Sample candidate API calls**
   The language model proposes possible tool calls within text.
2. **Execute the API calls**
   Tools such as a calculator or search engine are run to obtain results.
3. **Filter useful API calls**
   Keep only the calls that help the model better predict future tokens.
4. **Finetune the model**
   Train the model on the dataset augmented with useful API calls.



Step 1: Sample Candidate API Calls

Goal

Identify positions in text where calling a tool might help predict future tokens

Prompt the LM to insert API Calls, use in-context learning to teach the model the API syntax

Your task is to add calls to a Question Answering API.

Input:

Joe Biden was born in Scranton, Pennsylvania.

Output:

Joe Biden was born in

[QA("Where was Joe Biden born?")] Scranton,

[QA("In which state is Scranton?")] Pennsylvania.

Step 1: Sample Candidate API Calls (cont.)

Sample API Call Positions

For each token position i: $p_i = p_M(\langle API \rangle \mid P(x), x_{1:i-1})$

Keep positions where: $p_i > \tau_s$

Generate API Calls

For each selected position, sample up to m API calls $c_i^1, c_i^2, \ldots, c_i^m$

```
Original Text
      ↓
Prompt LM with API examples
      ↓
Predict <API> token probabilities
      ↓
Select high-probability positions
      ↓
Sample candidate API calls
```

The Nile has a length of [QA("What is the length of the Nile?")]

Result: Large set of candidate API calls embedded into text

Step 2: Executing Candidate API Calls

Goal

Obtain the actual outputs of the proposed API calls.

Execute each sampled API call

For each candidate call, $c = (a_c, i_c)$ where a_c = API name and i_c = API input; run the API to obtain result r

Tools used in Toolformer

Question Answering, Wikipedia Search, Calculator, Machine Translation, Calendar

Output

Text augment with API input and result

```
Candidate API Calls
        ↓
Execute Tools
        ↓
Return API Result
        ↓
Insert into Text
```

Step 3: Filtering Useful API Calls

Example	$L_i^- - L_i^+$	Useful
The Nile has an approximate length of <API> **QA(What is the approximate length of the Nile?)** → **6,853 km** </API> 6,853 kilometers, the White Nile being its main source.	2.08	✓
If Venus had an atmosphere similar to Earth's then you would expect Venus' mean temperature to be 499 K (1.74 x 287) rather than 735 K which is <API> **Calculator(735 / 499)** → **1.47** </API> 1.47 (735 / 499) times hotter than it should be.	1.59	✓
85 patients (23%) were hospitalised alive and admitted to a hospital ward. Of them, <API> **Calculator(85 / 23)** → **3.70** </API> 65% had a cardiac aetiology [...]	−0.02	✗
But hey, after the <API> **Calendar()** → **Today is Saturday, June 25, 2011.** </API> Disneyland fiasco with the fire drill, I think it's safe to say Chewey won't let anyone die in a fire.	−0.41	✗
The last time I was with <API> **QA(Who was last time I was with?)** → **The Last Time** </API> him I asked what he likes about me and he said he would tell me one day.	−1.23	✗

Problem

Not all generated API calls are useful

Idea

Keep API calls only if they improve language model predictions

Language Modeling Loss

$$L_i(z) = -\sum_{j=i}^{n} w_{j-i} \log p_M(x_j \mid z, x_{1:j-1})$$

For token prediction:

Weighted cross entropy loss measures how well the model predicts remaining tokens

Compare Two Scenarios

WIthout API result $L_i^- = \min(L_i(\varepsilon), L_i(e(c_i, \varepsilon)))$

With API result $L_i^+ = L_i(e(c_i, r_i))$

Keep API Call if $L_i^- - L_i^+ \geq \tau_f$

API result reduces prediction loss

Result

Filtered dataset: text + useful API calls

This becomes training set C*

Step 4: Fine Tuning the Toolformer Model

Construct Augmented Dataset C*

original dataset: $x = (x_1, x_2, ..., x_n)$

augmented with API calls: $x^* = (x_{1:i-1}, e(c_i, r_i), x_{i:n})$

c_i = API call, r_i = API response, $e(c_i, r_i)$ = serialized API call

Fine Tuning Objective

Train model using standard language modeling objective

$$\max \sum_t \log p(x_t \mid x_{<t})$$

Dataset C* contains the same text as original dataset **C** plus **helpful API calls**.

This preserves the model's **general language ability**.

```
Original Dataset C
        ↓
Add Useful API Calls
        ↓
Augmented Dataset C*
        ↓
Finetune Language Model
        ↓
Toolformer
```

Inference: Using tools during generation

Generation with Toolformer

Toolformer generates text as a normal language model until an API call is produced.

Step 1 — Generate API Call

When the model outputs: [Calculator(400 / 1400) → the system detects that a tool is required.

Step 2 — Execute API

Pause decoding and run the API. Example: Calculator(400 / 1400) = 0.29

Step 3 — Insert Result

The output is inserted into the sequence: [Calculator(400 / 1400) → 0.29]

Step 4 — Continue Generation

The model resumes generation using the result. Example: Out of 1400 participants, 400 (or 29%) passed the test.

API calls are represented as text tokens in the sequence.

End to End Example: Toolformer Pipeline

Step 1 Original Sentence

Out of 500 students, 125 passed the exam. Goal: predict 25%

Token sequence: Out | of | 500 | students | , | 125 | passed | the | exam

Step 2 All Token Positions Considered

Every position is a potential API insertion point. Based on our threshold, we would've chose after "125"

Position	Prefix	P(<API>)
1	Out	0.01
3	Out of 500	0.04
6	Out of 500 students, 125	**0.62**
8	passed the	0.02

Select positions where:

$$p_i > \tau_s$$

End to End Example: Toolformer Pipeline

Step 3 — Sample Candidate API Calls

From the selected position we sample k candidates.

Example:

- Candidate 1 (good) ; [Calculator(125 / 500)]
- Candidate 2 (bad) ; [QA("How many students passed the exam?")]

Step 4 – Execute APIs

Good call: Calculator(125 / 500) → 0.25

Bad call: QA(...) → "125 students passed"

Step 5 – Loss Filtering

Compare prediction loss for the continuation

With API: $L_i^+ = -\sum \log p(x_j | context + API\ result)$

Without API: No API Call at all $L_i(\epsilon)$ Or API call but no response $L_i(e(c_i, \epsilon))$ take better of two $L_i^- = \min(L_i(\epsilon), L_i(e(c_i, \epsilon)))$

End to End Example: Toolformer Pipeline

Final decision rule

if the api reduces the loss enough then this API call is considered useful $L_i^- - L_i^+ \geq \tau_f$

Case A Baseline: Computed with no API to test whether API result is useful; $L_i(\epsilon)$

Context: Out of 500 students, 125 (output: passed the exam)

Case B: API syntax but no result to check if syntax alone helps $L_i(e(c_i, \epsilon))$

Context: Out of 500 students, 125 [Calculator(125 / 500) →] (output: passed the exam)

Case C: API with result L_i^+

Out of 500 students, 125 [Calculator(125 / 500) → 0.25]

Compute loss for both candidates; filter based on threshold

Candidate 1: [Calculator(125 / 500)] ✔ KEEP

Candidate 2: [QA("How many students passed the exam?")] ✘ DISCARD

Experimental Setup

Base Model Toolformer is built on: **GPT-J (6.7B parameters)**

Baselines: GPT-J (no tools), GPT-J + CCNet fine tuning, OPT (66B), GPT-3 (175B)

Training Dataset

Language modeling dataset: CCNet corpus

Dataset augmentation: API calls automatically inserted, Filtered using loss reduction

Final dataset: **C* = CCNet + useful API calls**

Tools Available to the Model

All experiments are performed in zero shot settings

Toolformer (disabled)

GPT-J trained on API dataset but API calls disabled

isolates effect of tool usage

Your task is to add calls to a Question Answering API to a piece of text. The questions should help you get information required to complete the text. You can call the API by writing "[QA(question)]" where "question" is the question you want to ask. Here are some examples of API calls:

Input: Joe Biden was born in Scranton, Pennsylvania.

Output: Joe Biden was born in [QA("Where was Joe Biden born?")] Scranton, [QA("In which state is Scranton?")] Pennsylvania.

Input: Coca-Cola, or Coke, is a carbonated soft drink manufactured by the Coca-Cola Company.

Output: Coca-Cola, or [QA("What other name is Coca-Cola known by?")] Coke, is a carbonated soft drink manufactured by [QA("Who manufactures Coca-Cola?")] the Coca-Cola Company.

Input: x

Output:

API Name	Example Input	Example Output
Question Answering	Where was the Knights of Columbus founded?	New Haven, Connecticut
Wikipedia Search	Fishing Reel Types	Spin fishing > Spin fishing is distinguished between fly fishing and bait cast fishing by the type of rod and reel used. There are two types of reels used when spin fishing, the open faced reel and the closed faced reel.
Calculator	27 + 4 * 2	35
Calendar	ε	Today is Monday, January 30, 2023.
Machine Translation	sûreté nucléaire	nuclear safety

Results: Mathematical Reasoning

Toolformer significantly improves performance on math tasks.

Key findings:

- Calculator tool dramatically improves accuracy
- Toolformer outperforms GPT-J baseline by 3–5×
- Even surpasses GPT-3 (175B) on these tasks
- Model calls the calculator for ~98% of examples

Model	ASDiv	SVAMP	MAWPS
GPT-J	7.5	5.2	9.9
GPT-J + CC	9.6	5.0	9.3
Toolformer (disabled)	14.8	6.3	15.0
Toolformer	**40.4**	**29.4**	**44.0**
OPT (66B)	6.0	4.9	7.9
GPT-3 (175B)	14.0	10.0	19.8

Results: Factual Knowledge & QA

Toolformer improves factual knowledge tasks.

Key insights:

- Model learns to call question answering tools
- Can retrieve missing factual information
- Large improvements over GPT-J baseline
- Competitive with much larger models

Model	WebQS	NQ	TriviaQA
GPT-J	18.5	12.8	43.9
GPT-J + CC	18.4	12.2	45.6
Toolformer (disabled)	18.9	12.6	46.7
Toolformer	**26.3**	**17.7**	**48.8**
OPT (66B)	18.6	11.4	45.7
GPT-3 (175B)	29.0	22.6	65.9

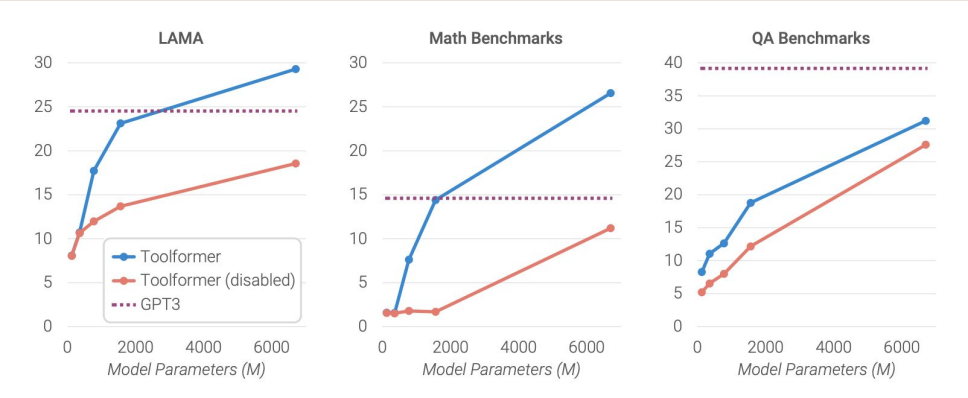
Important stat: Toolformer calls the QA tool in ~99% of examples

.

When Do Models Learn to Use Tools?

Tool use is an emergent capability of larger models.

Scaling behavior: tool use vs model size



Key observations:

- Small models cannot effectively use tools
- Tool usage improves starting around ~775M parameters
- Larger models learn when and how to call APIs
- Performance gap between with tools vs without tools grows with scale

Limitations of Toolformer

1. No Tool Chaining

   - Toolformer cannot combine multiple tools in sequence
   - Example: search → calculator → answer

2. Limited Interaction with Tools

   - Cannot iteratively refine queries
   - Cannot browse multiple search results

3. Sample Inefficiency

   - Requires processing large datasets to obtain useful tool calls
   - Some tools generate very few useful examples

4. Sensitivity to Prompts

   - Tool usage decisions depend heavily on input wording

Key Contributions & Takeaways

Self-Supervised Tool Learning

- Introduces a method for LMs to learn tool usage without labeled data
- Model generates and filters its own training examples

Small model + tools can outperform much larger models; External APIs improve tasks like:

- arithmetic
- factual retrieval
- translation
- temporal reasoning

Moving Toward Language Model Agents

- Demonstrates that LMs can decide when to use external tools
- Important step toward tool-using AI agents

Reflexion: Language Agents with Verbal Reinforcement Learning

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, Shunyu Yao

Motivation: Stateless Agents

- ReAct and Toolformer are powerful, but both are **stateless across episodes**
- Each new attempt starts completely fresh, with no memory of prior failures
- Example from Paper: An ALFWorld agent needs to find a pepper shaker and put it in a drawer. It *thinks* it has picked up the item but the observation never confirmed this. It proceeds through 20+ actions trying to complete a task while holding nothing. It fails. It resets. It makes the same mistake again.

Traditional RL doesn’t solve this because:

- Requires thousands of training samples
- Requires differentiable reward signals
- Expensive weight updates — can't be done on the fly
- Black-box: you can't read *what* the agent learned

Learning from Mistakes

A strong LLM can read its own failed trajectory, diagnose the mistake, write an actionable lesson, and carry that lesson into the next attempt. Random retry has no directional signal. Reflection provides a **specific, targeted correction.** The agent isn't re-sampling blindly.

Traditional RL	Reflexion
Scalar reward signal	Verbal summary of what went wrong
Learning via backpropagation	Learning via text stored in context
Requires many samples + gradient computation	Requires one capable LLM + a few trials
Opaque — you can't read the update	Transparent — the "gradient" is readable English
Permanent — changes model weights	Ephemeral — lives in the context window

Components of Reflexion

Actor

- The agent that generates actions, typically **ReAct** (thought + action interleaved)
- Also reads from the memory buffer on each trial
- At each step: conditioned on current state + past reflections

Evaluator

- Scores each completed trajectory
- Three variants used across tasks:
  - Binary environment signal (task done / not done)
  - Hand-written heuristic (e.g., if agent repeats same action 3× → stuck)
  - LLM-as-judge for open-ended tasks

Self-Reflection Model

- An LLM prompted to analyze: failed trajectory + evaluator signal → verbal lesson
- Produces a natural language post-mortem: what went wrong, why, what to do differently
- Output stored in memory buffer for the next trial

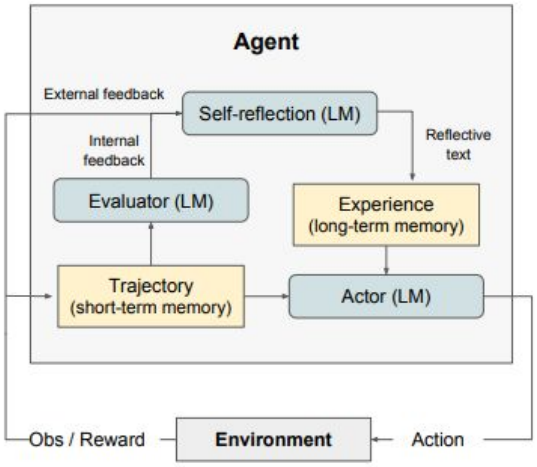


Figure 2: (a) Diagram of Reflexion. (b

Components of Reflexion

Memory

- Stores verbal reflection of error, accessed each time by the actor

Process

- An LLM prompted to analyze: failed trajectory + evaluator signal → verbal lesson
- Produces a natural language post-mortem: what went wrong, why, what to do differently
- Output stored in memory buffer for the next trial

Algorithm 1 Reinforcement via self-reflection

Initialize Actor, Evaluator, Self-Reflection: M_a, M_e, M_{sr}
Initialize policy $\pi_\theta(a_i|s_i)$, $\theta = \{M_a, mem\}$
Generate initial trajectory using π_θ
Evaluate τ_0 using M_e
Generate initial self-reflection sr_0 using M_{sr}
Set $mem \leftarrow [sr_0]$
Set $t = 0$
while M_e not pass or $t <$ max trials **do**
  Generate $\tau_t = [a_0, o_0, \ldots a_i, o_i]$ using π_θ
  Evaluate τ_t using M_e
  Generate self-reflection sr_t using M_{sr}
  Append sr_t to mem
  Increment t
end while
return

Example: Decision, Programs, Reasoning

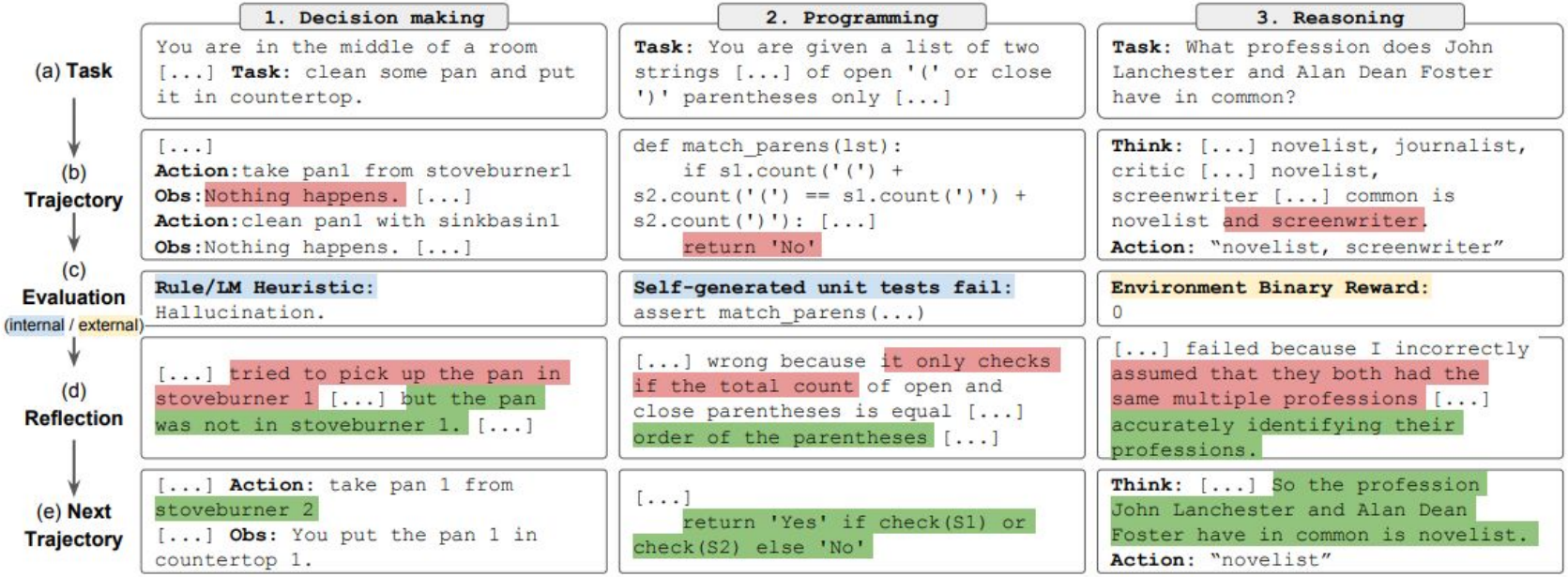


Figure 1: Reflexion works on decision-making 4.1, programming 4.3, and reasoning 4.2 tasks.

Reflexion: Language Agents with Verbal Reinforcement Learning

Related work on reasoning and decision-making					
Approach	**Self refine**	**Hidden constraints**	**Decision making**	**Binary reward**	**Memory**
Self-refine [15]	✓	✗	✗	✗	✗
Beam search [27]	✓	✓	✓	✓	✗
Reflexion (ours)	✓	✓	✓	✓	✓

Related work on programming					
Approach Test execution	**Test execution**	**Debugging**	**Self-generated tests**	**Multiple languages**	**Self-reflection**
AlphaCode [14]	✓	✗	✗	✓	✗
CodeT [5]	✓	✗	✓	✗	✗
Self-debugging [7]	✓	✓	✗	✗	✗
CodeRL [12]	✓	✓	✗	✗	✗
Reflexion (ours)	✓	✓	✓	✓	✓

Reflexion on ALFWorld:

134 AlfWorld tasks across 6 categories (find objects, move objects, manipulate objects)

Method	Tasks Solved (out of 134)	Notes
ReAct only	~94/134 (~70%)	Plateaus after trial 6, hallucination rate stays at 22%
ReAct + Reflexion (heuristic)	**130/134 (97%)**	Improves steadily across 12 trials
ReAct + Reflexion (LLM eval)	~125/134 (93%)	Slightly lower — LLM evaluator less reliable than heuristic

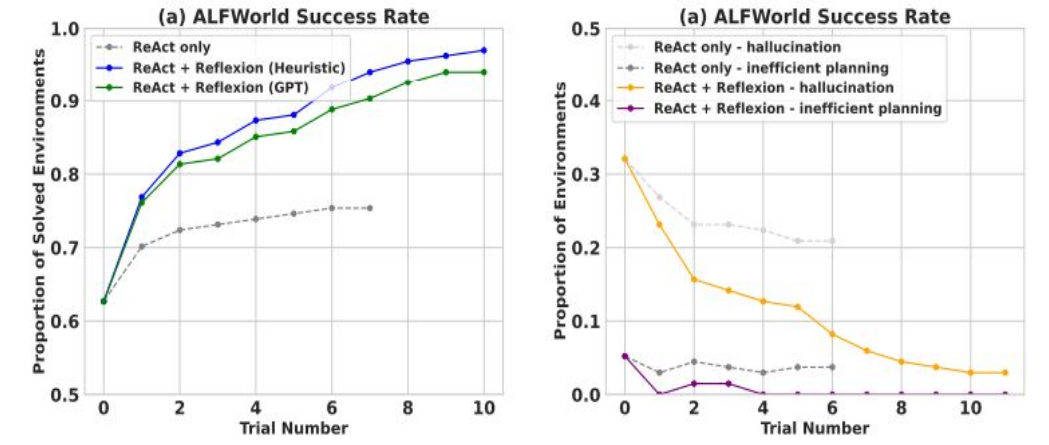


Figure 3: (a) AlfWorld performance across 134 tasks showing cumulative proportions of solved tasks using self-evaluation techniques of (Heuristic) and (GPT) for binary classification. (b) Classification of AlfWorld trajectories by reason of failure.

- Sharp jump between trial 1 → trial 2 (first reflections applied immediately)
- Steady improvement trials 2–12
- ReAct alone: flatlines after trial 6 — stuck at a ceiling it cannot escape

Results

Reflexion can solve 2 problems:

Phantom possession errors — agent thinks it has an item it doesn't have, executes a long wrong trajectory

- Reflection identifies this immediately: *"I assumed I had the item but never confirmed it"*

Exploration failures — too many surfaces to search, agent gives up

- Reflection accumulates spatial knowledge over trials: *"I've now checked shelves 1-4, so next time start with 5-6"*

Reflexion builds a progressive model of the environment across trials, accumulating lessons that compound over time

HotPotQA

HotPotQA [28] is a Wikipedia-based dataset with 113k question-and-answer pairs that challenge agents to parse content and reason over several supporting documents.

- 100 HotPotQA questions (multi-hop Wikipedia reasoning)
- Two agent types: CoT (reasoning only) and ReAct (reasoning + search)
- Memory: last 3 reflections

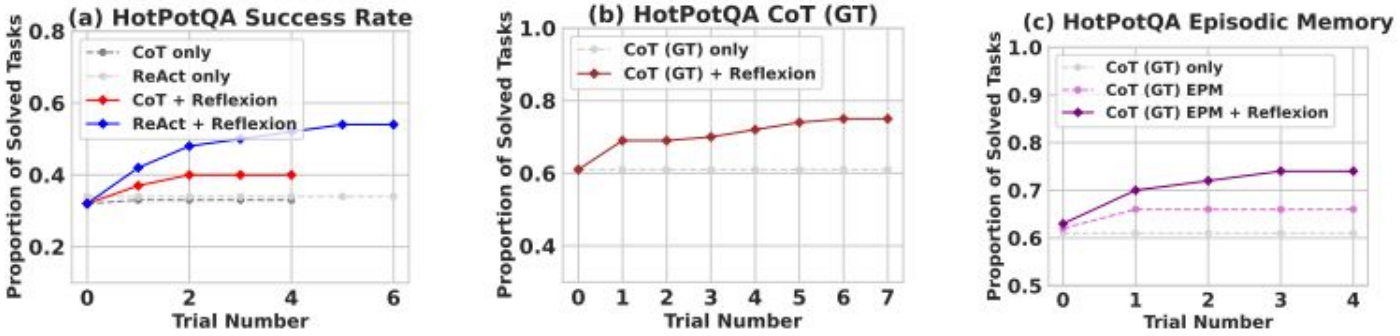


Figure 4: Chain-of-Thought (CoT) and ReAct. Reflexion improves search, information retrieval, and reasoning capabilities on 100 HotPotQA questions. (a) Reflexion ReAct vs Reflexion CoT (b) Reflexion CoT (GT) for reasoning only (c) Reflexion vs episodic memory ablation.

HotPotQA: Results

- Overall: **+20% improvement** over baselines
- Episodic memory alone (just storing what happened) helps: **+6%**
- Adding verbal self-reflection on top helps further: **+8%**
- **The verbal analysis step has independent value beyond just remembering**

It's not enough to remember that you failed. You need to articulate *why* you failed and *what to do differently*. The act of generating the verbal explanation is doing real cognitive work — it's not just a retrieval mechanism.

Method	Accuracy
CoT baseline	~33%
ReAct baseline	~39%
CoT + Reflexion	~53%
ReAct + Reflexion	**~58%**

Condition	Accuracy
CoT-GT alone	55%
CoT-GT + Episodic Memory (store trajectory, no reflection)	61%
CoT-GT + Full Reflexion (store trajectory + verbal reflection)	**69%**

Reflexion Code: Surpassing GPT-4

- Benchmarks: HumanEval, MBPP (Python + Rust), LeetcodeHard (new benchmark)
- Actor: GPT-4 generating Python/Rust function implementations
- Self-evaluation: model writes its own unit tests, runs them, reflects on failures
- Memory: last 1 reflection (code context is long)

It's not enough to remember that you failed. You need to articulate *why* you failed and *what to do differently*. The act of generating the verbal explanation is much more important than just having a retrieval mechanism.

Benchmark + Language	Prev SOTA Pass@1	SOTA Pass@1	Reflexion Pass@1
HumanEval (PY)	65.8 (CodeT [5] + GPT-3.5)	80.1 (GPT-4)	**91.0**
HumanEval (RS)	–	60.0 (GPT-4)	**68.0**
MBPP (PY)	67.7 (CodeT [5] + Codex [6])	**80.1** (GPT-4)	77.1
MBPP (RS)	–	70.9 (GPT-4)	**75.4**
Leetcode Hard (PY)	–	7.5 (GPT-4)	**15.0**

Table 1: Pass@1 accuracy for various model-strategy-language combinations. The base strategy is a single code generation sample. All instruction-based models follow zero-shot code generation.

Reflexion Code: Surpassing GPT-4

Why programming is a special case:

- Unlike decision-making tasks, programming has **grounded self-evaluation**: run the code, check if it passes tests
- Reflexion generates its own unit test suite (up to 6 tests), runs them against its solution, reflects on failures
- This is fully automated test-driven development

Write function → Run self-generated tests → Tests fail → Reflect: "The function fails on edge case where input is empty list. I forgot to handle the base case. I should add a check for len(input)==0" → Rewrite function → Tests pass → Submit

Why MBPP underperforms:

- **False positive unit tests**: on MBPP, 16.3% of the time self-generated tests pass on an *incorrect* solution
- Agent incorrectly declares success and submits a wrong answer
- On HumanEval this rate is only 1.4% — explains the difference
- **Lesson:** Reflexion is only as good as its self-evaluation step

Ablation: Removing Each Component

Without test generation:

- Model has no objective signal about whether its code is correct
- Without knowing it's wrong, it participates in all iterations, making unnecessary edits to working code
- Performance *drops below baseline* — harmful over-editing

Without verbal self-reflection:

- Model knows tests failed but must jump directly from "test X failed" to "here is my fix"
- Cannot bridge the gap between error identification and implementation improvement
- Matches baseline — test execution alone adds nothing on hard problems

Approach	Test Generation	Self-reflection	Pass@1 (Acc)
Base model	False	False	0.60
Test generation omission	False	True	0.52
Self-reflection omission	True	False	0.60
Reflexion	True	True	**0.68**

Table 3: Pass@1 accuracy for various compromised approaches on the Reflexion approach using GPT-4 as the base model on HumanEval Rust - 50 hardest problems

Reflexion: Limitations

Limitation 1: Context window cap

- Memory is bounded to 1–3 past reflections to fit in context
- Long histories require external memory (vector databases) which has not yet been implemented
- Implication: lessons learned in trial 1 may be forgotten by trial 10

Limitation 2: Model quality dependency

- Self-reflection is an **emergent capability** — only works for strong, large models
- Paper shows: starchat-beta (smaller open model) gets **zero improvement** from Reflexion
- Baseline and Reflexion accuracy are identical for weak models — they cannot reliably diagnose their own mistakes
- Reflexion amplifies capability; it cannot create capability that isn't there

Limitation 3: Self-evaluation reliability

- The whole framework depends on the Evaluator correctly identifying failure
- The Self-Reflection model must correctly identify *why* it failed (credit assignment)
- If the model misdiagnoses its mistake, it conditions the next attempt on a wrong lesson — potentially making things worse

Reflexion: Limitations & Future Work

Limitation 4: Local optima in verbal RL

- Like any optimization, verbal policy optimization can converge on a locally good but globally wrong strategy
- No formal guarantee of convergence or correctness

Open questions for future work:

- Can reflections be distilled back into **weight updates**; combining verbal interpretability with permanent learning?
- Can agents reflect on **each other's** failures in a multi-agent setting?
- How do you build **long-term memory** that retrieves the right past reflection for the current situation?
- Can self-evaluation be made more **calibrated and reliable**, especially for the false positive unit test problem?

Reflexion Contributions & Conclusions

- First framework to show LLMs can improve across episodes without weight updates
- Demonstrates that verbal self-analysis is computationally meaningful, not just introspective flavor text
- Achieves SOTA on HumanEval by doing something conceptually simple: write tests, run them, explain the failure, fix it
- Points toward a future where agents maintain persistent learned experience, not just skills, but wisdom from failure

Reflexion suggests that the path to more capable agents may not require bigger models or more training data, it may require giving agents the ability to reflect on and learn from their own experience, the same way humans do.

Connecting the Papers

ReAct (Reasoning + Acting):

- Combines reasoning with environment actions
- Allows language models to interact with external environments while reasoning about tasks

Toolformer (Tool Usage)

- Teaches language models when and how to call external tools
- Uses self-supervised training to learn API usage

Reflexion (Learning from Experience)

- Enables agents to improve across attempts
- Uses verbal self-reflection to learn from past mistakes

Together, these approaches extend language models beyond text generation toward more capable autonomous agents.

Thank you